

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук

должность, уч. степень, звание

подпись, дата

А. В. Бржезовский

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 5

ЗАПРОСЫ НА ЯЗЫКЕ SQL: ПОДЗАПРОСЫ

по курсу:

МЕТОДЫ И СРЕДСТВА ПРОЕКТИРОВАНИЯ ИНФОРМАЦИОННЫХ
СИСТЕМ И ТЕХНОЛОГИЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: изучение применения подзапросов и операторов EXISTS, NOT EXISTS при выполнении запросов и операций манипулирования данными.

2 Постановка задачи

Требуется реализовать запросы ж-и варианта задания с использованием подзапросов и операторов EXISTS и NOT EXISTS, продемонстрировать использование подзапросов в операторах INSERT, UPDATE, DELETE, а также сравнить выполнение запросов с EXISTS и теоретико-множественными операциями при наличии NULL-значений.

Вариант: 4.

Создайте базу данных для хранения следующих сведений: студент, группа, дисциплина, преподаватель, лабораторная работа, рейтинг за сданную лабораторную работу.

Составить запросы на:

1. лабораторные по БД, которые нужно досдать Сыроежкину из группы 4000;
2. студентов, получивших одинаковый рейтинг за все работы по БД;
3. студентов, сдавших все работы по БД.

3 Ход выполнения

В ходе выполнения работы была использована ранее созданная база данных, содержащая сведения о студентах, дисциплинах, лабораторных работах и оценках. Сперва были проверены существующие данные и добавлены недостающие записи для корректного выполнения запросов.

Далее были реализованы запросы по варианту задания. На рисунке 1 приведен код запроса (1) и результат его выполнения (лабораторные работы по БД, которые необходимо досдать студенту по фамилии и группе).

```

27 -- ж) лабораторные по БД для досдачи студентом Тестовым из группы 4000
28 SELECT lw.lab_title
29 FROM lab_work lw
30 JOIN discipline d ON lw.discipline_id = d.discipline_id
31 WHERE d.discipline_name = 'Базы данных'
32 AND EXISTS (
33     SELECT *
34     FROM student s
35     JOIN student_group sg ON s.group_id = sg.group_id
36     WHERE s.last_name = 'Тестов'
37     AND sg.group_name = '4000'
38 )
39 AND NOT EXISTS (
40     SELECT *
41     FROM lab_rating lr
42     JOIN student s ON lr.student_id = s.student_id
43     JOIN student_group sg ON s.group_id = sg.group_id
44     WHERE s.last_name = 'Тестов'
45     AND sg.group_name = '4000'
46     AND lr.lab_id = lw.lab_id
47 );
48

```

	AZ lab_title	Value
1	ЛР8	1 ЛР8
2	ЛР2	

Рисунок 1 — Запрос ЛР на досдачу студентом

На рисунке 2 приведен код запроса (2) и результат его выполнения (студенты с одинаковыми оценками за все работы).

```

49 -- з) студенты с одинаковыми оценками за все работы
50 SELECT
51     s.student_id,
52     s.last_name
53 FROM student s
54 WHERE
55     EXISTS (
56         SELECT *
57         FROM lab_rating lr
58         WHERE
59             lr.student_id = s.student_id
60     )
61     AND NOT EXISTS (
62         SELECT *
63         FROM
64             lab_rating lr1
65             JOIN lab_rating lr2 ON lr1.student_id = lr2.student_id
66         WHERE
67             lr1.student_id = s.student_id
68             AND lr1.score <> lr2.score
69     );
70

```

	student_id	AZ last_name	Value
1	3	Петров	
2	5	Тестов	

Рисунок 2 — Запрос студентов с одинаковыми оценками за все работы

На рисунке 3 приведен код запроса (3) и результат его выполнения (студенты, сдавшие все лабораторные работы по БД).

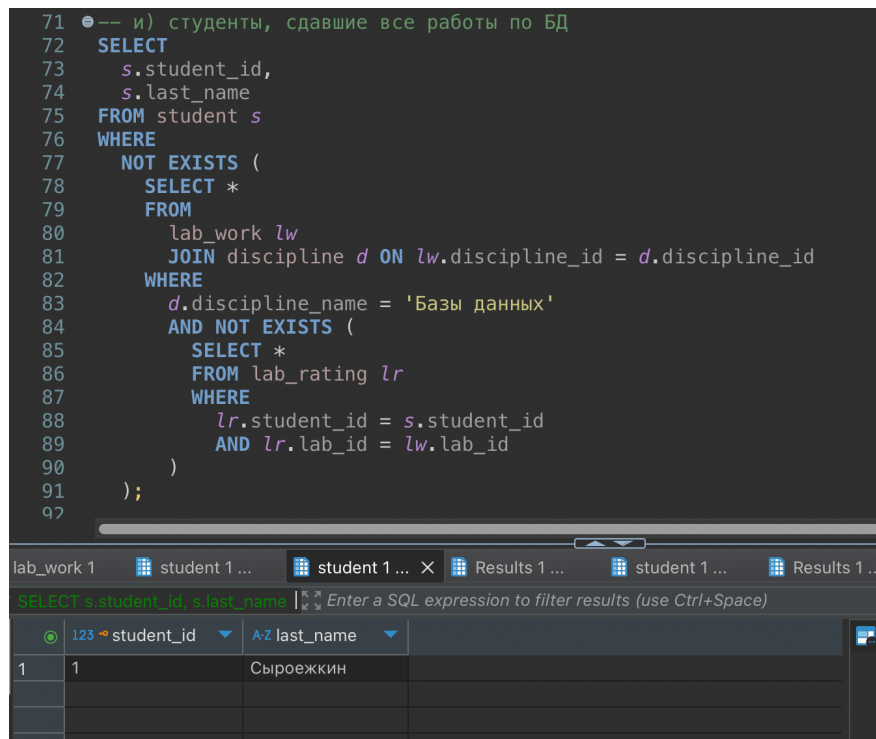


Рисунок 3 — Запрос студентов со всеми сданными работами по БД

После этого были разработаны запросы с использованием подзапросов в операторах изменения данных. На рисунке 4 приведен пример использования подзапроса в INSERT.

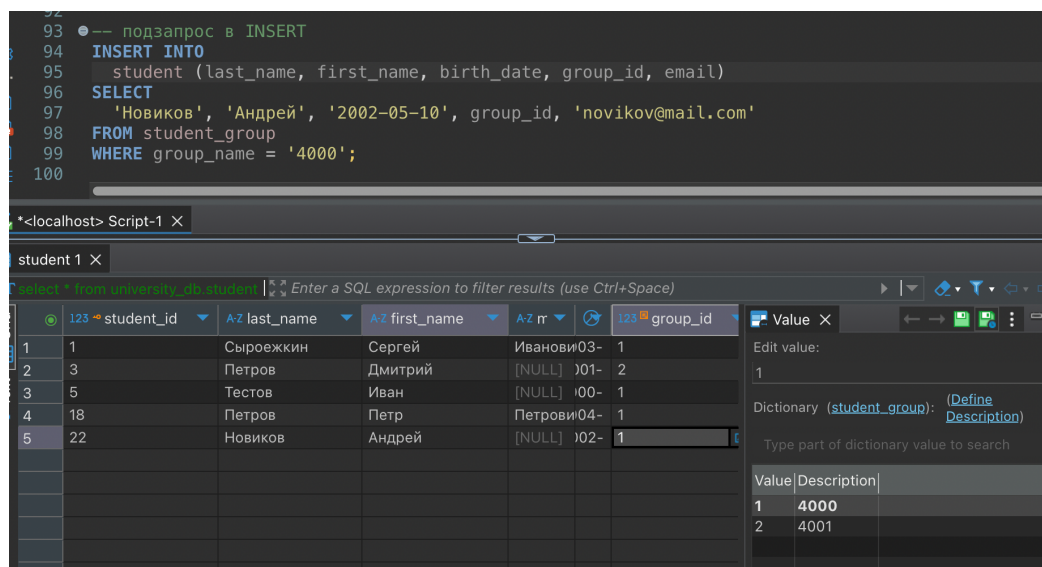


Рисунок 4 — Вставка в student с указанием номера группы в подзапросе

На рисунке 5 приведен пример использования подзапроса в UPDATE.

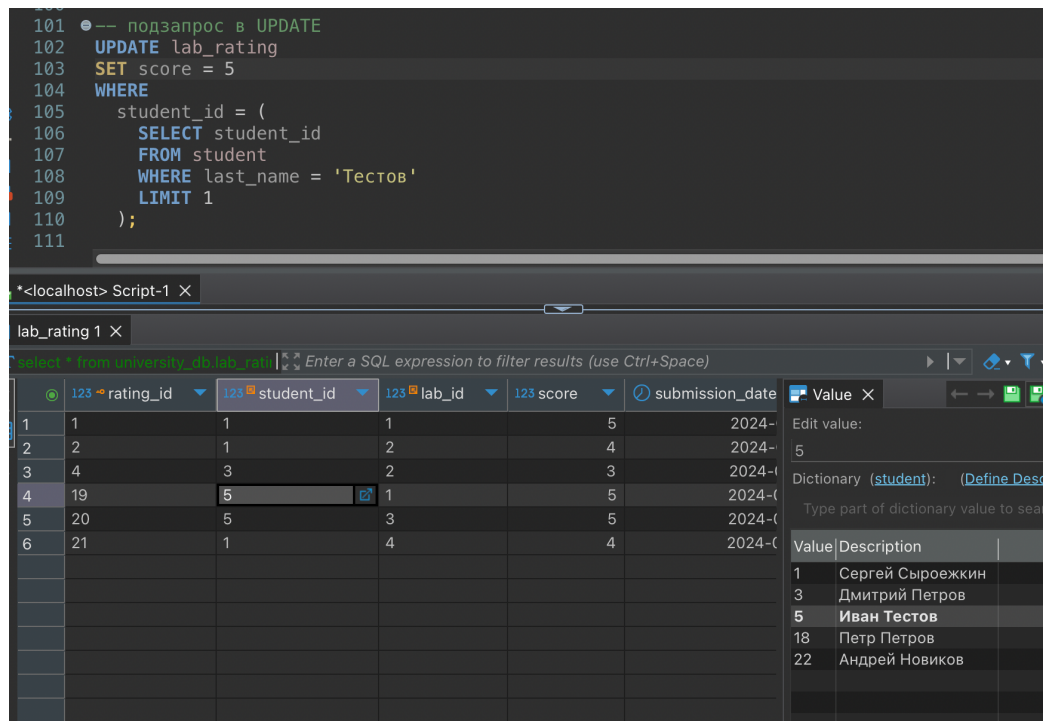


Рисунок 5 — Обновление первой по порядку работы студента Тестова

На рисунке 6 приведен пример использования подзапроса в DELETE.

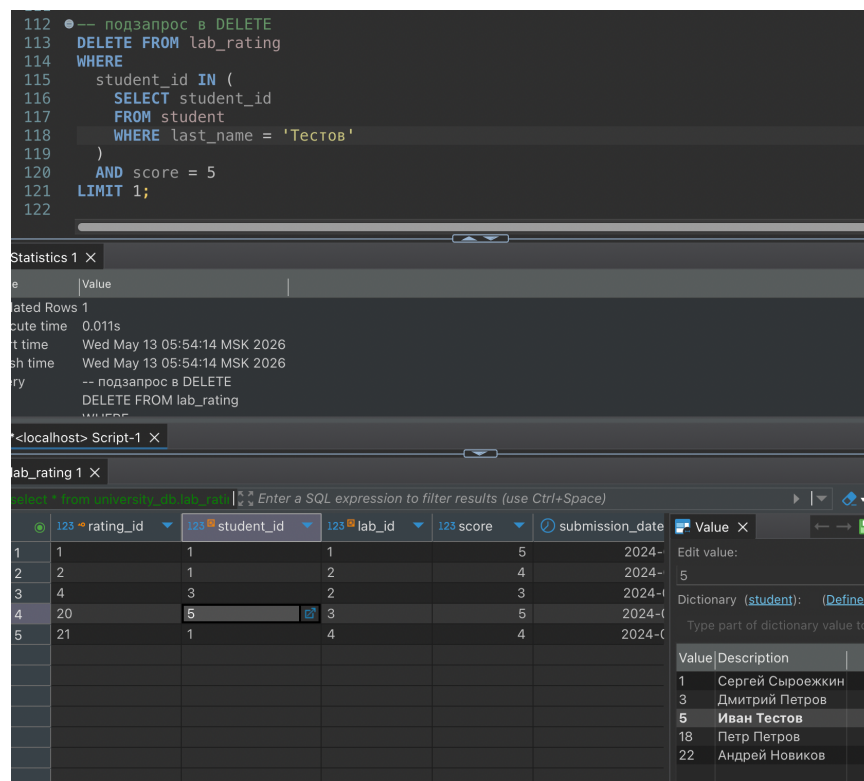


Рисунок 6 — Удаление первой по порядку работы с оценкой «5» студента Тестова

Далее были реализованы запросы с использованием EXISTS и NOT EXISTS, эквивалентные теоретико-множественным операциям INTERSECT и

EXCEPT. На рисунке 7 приведен пример реализации запроса пересечения (INTERSECT) для выбора одинаковых значений электронных почт студентов и преподавателей с учетом возможного наличия NULL-значений.

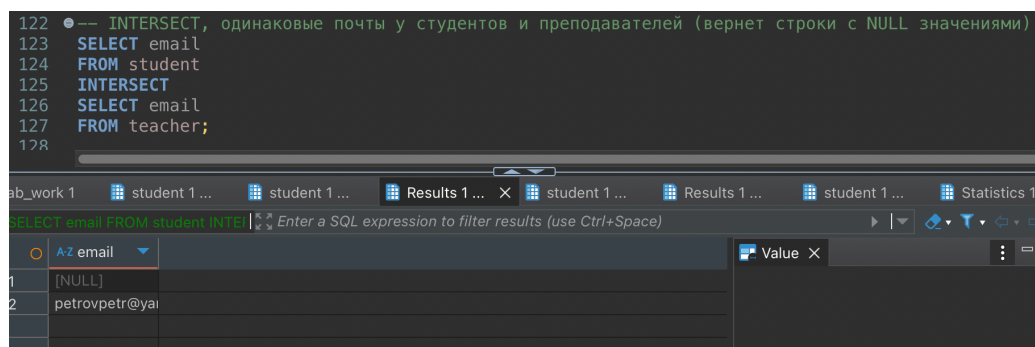


Рисунок 7 — Запрос пересекающихся email между студентами и преподавателями

На рисунке 8 приведен пример реализации аналогичного запроса с использованием EXISTS, позволяющего определить одинаковые почты студентов и преподавателей без некорректной обработки NULL.

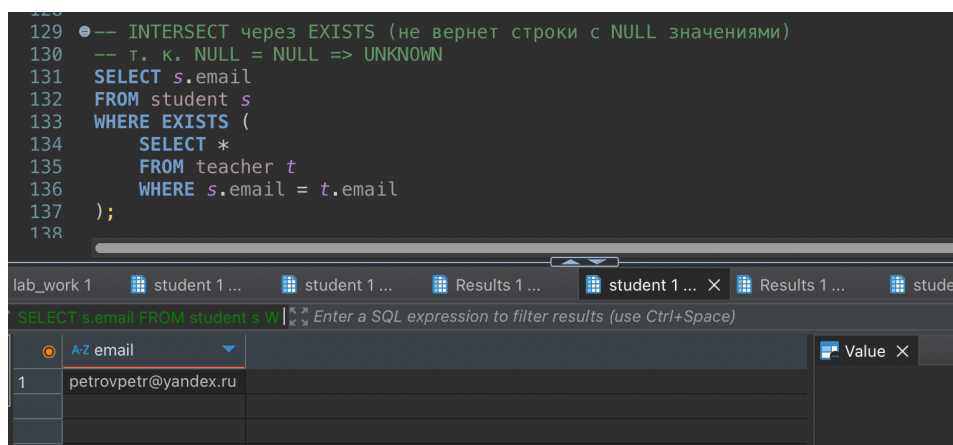


Рисунок 8 — Запрос пересекающихся email между студентами и преподавателями через EXISTS

На рисунке 9 приведен пример реализации разности (EXCEPT) для определения почт студентов, отсутствующих среди преподавателей, с учетом возможных NULL-значений.

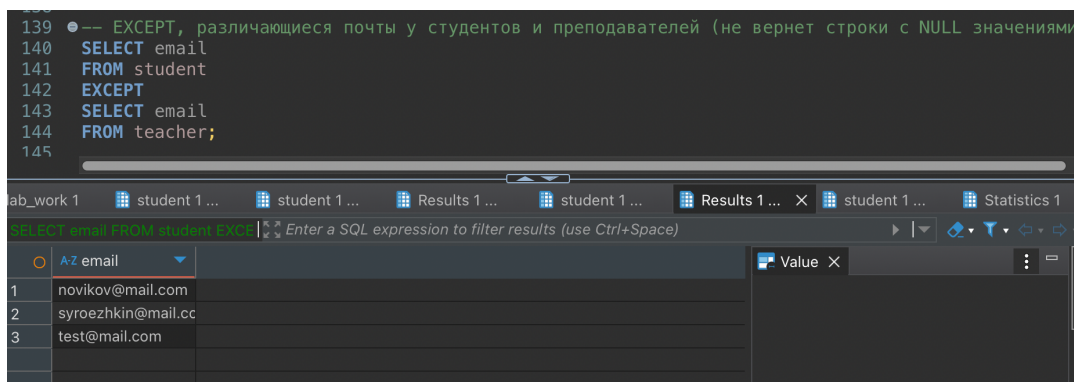


Рисунок 9 — Запрос почт студентов, исключая пересечения с почтами преподавателей

На рисунке 10 приведен пример реализации аналогичного запроса с использованием NOT EXISTS, позволяющего получить корректный результат при наличии NULL.

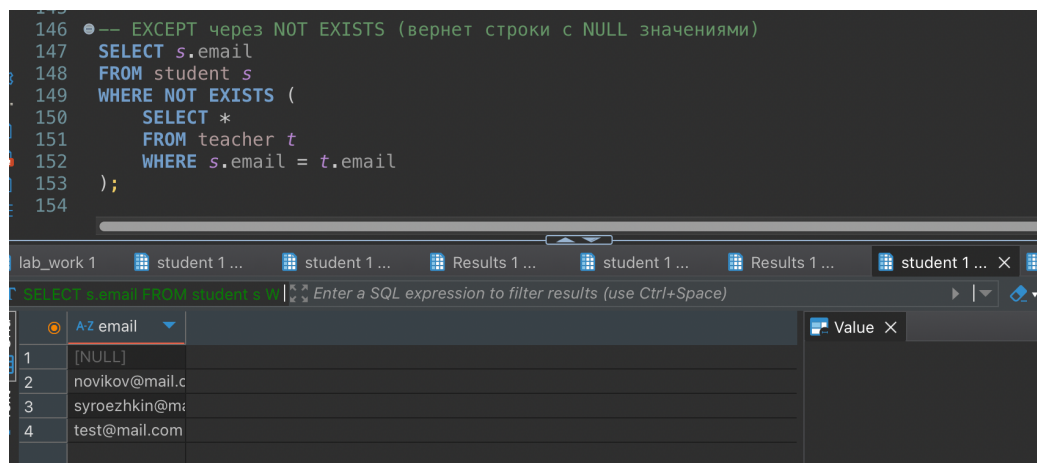


Рисунок 10 — Запрос почт студентов, исключая пересечения с почтами преподавателей через NOT EXISTS

Реализованный SQL-скрипт приведен в Приложении. В ходе анализа установлено, что теоретико-множественные операции INTERSECT и EXCEPT обрабатывают NULL-значения как обычные значения, то есть считают NULL = NULL истинным совпадением. В результате такие значения могут попадать в итоговый набор данных.

В то же время запросы с использованием EXISTS и NOT EXISTS основаны на операциях сравнения. При сравнении с NULL результат выражения становится UNKNOWN, что исключает такие строки из результата.

4 Выводы

В ходе выполнения работы были освоены подзапросы и операторы EXISTS и NOT EXISTS в SQL-запросах и операторах манипулирования данными. Были реализованы запросы для поиска связанных записей и записей, не имеющих соответствующих связей в других таблицах, а также продемонстрировано применение подзапросов в INSERT, UPDATE и DELETE.

Также было установлено, что различие между INNERSECT, EXCEPT и эквивалентными запросами через (NOT) EXISTS связано с особенностями обработки NULL. Теоретико-множественная операция рассматривает NULL как обычное значение и выполняет сравнение по принципу равенства, тогда как (NOT) EXISTS использует предикатное сравнение, в котором NULL = NULL дает значение UNKNOWN. Это приводит к различиям в результирующих наборах данных при наличии NULL-значений.

ПРИЛОЖЕНИЕ

```
USE university_db;

-- добавление недостающих данных

/*INSERT INTO
  lab_rating (student_id, lab_id, score, submission_date)
VALUES
  (5, 1, 3, '2024-01-20');

INSERT INTO
  lab_rating (student_id, lab_id, score, submission_date)
VALUES
  (5, 3, 2, '2024-01-25');*/

/*INSERT INTO
  lab_rating (student_id, lab_id, score, submission_date)
VALUES
  (1, 4, 4, '2024-01-30');*/

/*
INSERT INTO student
  (last_name, first_name, middle_name, birth_date, group_id, email, phone)
VALUES
  ('Петров', 'Петр', 'Петрович-Младший', '2004-07-02', 1, 'petrovpetr@yandex.ru',
  NULL);
*/

-- ж) лабораторные по БД для досдачи студентом Тестовым из группы 4000
SELECT lw.lab_title
FROM lab_work lw
JOIN discipline d ON lw.discipline_id = d.discipline_id
WHERE d.discipline_name = 'Базы данных'
AND EXISTS (
  SELECT *
  FROM student s
  JOIN student_group sg ON s.group_id = sg.group_id
  WHERE s.last_name = 'Тестов'
    AND sg.group_name = '4000'
)
AND NOT EXISTS (
  SELECT *
  FROM lab_rating lr
  JOIN student s ON lr.student_id = s.student_id
  JOIN student_group sg ON s.group_id = sg.group_id
  WHERE s.last_name = 'Тестов'
    AND sg.group_name = '4000'
    AND lr.lab_id = lw.lab_id
);

-- з) студенты с одинаковыми оценками за все работы
SELECT
  s.student_id,
  s.last_name
FROM student s
WHERE
  EXISTS (
    SELECT *
    FROM lab_rating lr
    WHERE
      lr.student_id = s.student_id
```

```

)
AND NOT EXISTS (
  SELECT *
  FROM
    lab_rating lr1
  JOIN lab_rating lr2 ON lr1.student_id = lr2.student_id
  WHERE
    lr1.student_id = s.student_id
    AND lr1.score <> lr2.score
);

-- и) студенты, сдавшие все работы по БД
SELECT
  s.student_id,
  s.last_name
FROM student s
WHERE
  NOT EXISTS (
    SELECT *
    FROM
      lab_work lw
    JOIN discipline d ON lw.discipline_id = d.discipline_id
    WHERE
      d.discipline_name = 'Базы данных'
      AND NOT EXISTS (
        SELECT *
        FROM lab_rating lr
        WHERE
          lr.student_id = s.student_id
          AND lr.lab_id = lw.lab_id
      )
  );

-- подзапрос в INSERT
INSERT INTO
  student (last_name, first_name, birth_date, group_id, email)
SELECT
  'Новиков', 'Андрей', '2002-05-10', group_id, 'novikov@mail.com'
FROM student_group
WHERE group_name = '4000';

-- подзапрос в UPDATE
UPDATE lab_rating
SET score = 5
WHERE
  student_id = (
    SELECT student_id
    FROM student
    WHERE last_name = 'Тестов'
    LIMIT 1
  );

-- подзапрос в DELETE
DELETE FROM lab_rating
WHERE
  student_id = (
    SELECT student_id
    FROM student
    WHERE last_name = 'Тестов'
  )
  AND score = 5
LIMIT 1;

```

```

-- INTERSECT, одинаковые почты у студентов и преподавателей (вернет строки с NULL
значениями)
SELECT email
FROM student
INTERSECT
SELECT email
FROM teacher;

-- INTERSECT через EXISTS (не вернет строки с NULL значениями)
-- т. к. NULL = NULL => UNKNOWN
SELECT s.email
FROM student s
WHERE EXISTS (
    SELECT *
    FROM teacher t
    WHERE s.email = t.email
);

-- EXCEPT, почты студентов с исключением пересекающихся почт преподавателей (не
вернет строки с NULL значениями)
SELECT email
FROM student
EXCEPT
SELECT email
FROM teacher;

-- EXCEPT через NOT EXISTS (вернет строки с NULL значениями)
SELECT s.email
FROM student s
WHERE NOT EXISTS (
    SELECT *
    FROM teacher t
    WHERE s.email = t.email
);

```